TreeSync Manual

TreeSync syncs two directory trees and checks every copy by SHA-256. It is four programs, in the **TreeSyncApp** folder of your app list:

- **TreeSyncBuild**: builds and tests it, with bsir's requirements check.
- **TreeSyncInstall**: installs the GCC and / or Clang release build for root, or removes it.
- **TreeSyncRoot-GCC**, **TreeSyncRoot-Clang**: sync as root, any folder (administrators).
- **TreeSyncHome-GCC**, **TreeSyncHome-Clang**: sync as you, inside your home (anyone).

A command-line program, `treesync-cli`, does the actual work; the two sync GUIs start it and show what it does.

This manual takes you from a fresh Linux machine to a running sync:

Part	What it covers
1. Bootstrap	Get the code, install the compilers and libraries, get U++, first build
2. Build	`build.sh`, TreeSyncBuild, debug and release, tests, `verify.sh` and the pre-push hook, TreeSyncInstall
3. Run	TreeSyncRoot and TreeSyncHome step by step, the command line, remote trees, root, checking and attributes
4. Files and troubleshooting	Where everything is kept, and what to do when something goes wrong

> **Warning.** TreeSync is experimental testing software. It copies, overwrites and deletes files. Try it on copies first, keep backups, and use **Dry run** before a real run. It is provided "AS IS" under the Apache License 2.0 (sections 7 and 8).

In TreeSync, the **Help** menu opens these chapters as PDFs, and **F1** opens the one for the program you are in (TreeSyncRoot and TreeSyncHome: Run; TreeSyncBuild and TreeSyncInstall: Build).

Rebuilding the PDFs. The `.md` files here are the sources. After editing them run `./build-pdf.sh` in this folder: it writes `pdf/`, one PDF per chapter and `TreeSync-Manual.pdf` with all of them (it needs python3 with its markdown module, Chrome or Chromium, and pdfunite).

Quick start

For someone who has done this before. Each step is explained in the parts above.

```
# 1. Get the code (dev-work, the default branch)
git clone https://github.com/JayACarlsonAtHome/jac317.git
cd jac317/v001

# 2. Check the machine, install what is missing (it asks first), build and test
./bsir.sh

# 3. Build the release programs (with all tests, both compilers)
./verify.sh

# 4. Install them for root (asks for your sudo password)
./install.sh

# 5. Start TreeSync: "TreeSync" in the desktop's app list, or
treesync
```

The installed TreeSync syncs as root for administrators (members of wheel); you build as yourself in `v001/`, and run `./install.sh` again after each build you want to use.

In the GUI: pick a **source** and a **target** folder, check what to sync in the source tree, then **Sync > Dry run** (Shift+F9) to see what would happen and **Sync > Run** (F9) to do it.

Words used in this manual

Word	Meaning
Source	The tree copied from
Target	The tree copied to
Copy / Mirror / Merge	The three modes: add and update only; make the target match the source (removing extras); copy changes both ways
Dry run	A run that only reports what it would do and changes nothing
Safe delete	Instead of deleting or overwriting, TreeSync moves the old file into `<target>.old`, or into `<target>/.treesync-old` when the target is the top folder of a disk (on by default)
Run	One sync, with a number; it is recorded in the database and can be resumed if stopped
Scan only	Reads and hashes both trees and compares them; changes nothing
`v001/`	The folder that holds the code, the scripts and (after a build) `out/` with the programs

1. Bootstrap: from a fresh machine to a first build

`v001/bsir.sh` (Bootstrap, Setup, Install & Run) takes a Linux machine from nothing to a built and tested TreeSync. It is safe to run again at any time: it runs the system update every time, and otherwise leaves alone whatever is already there.

1.1 What you need

- **Linux**: RHEL 9 and its rebuilds (Rocky, Alma), Fedora, or Linux Mint / Ubuntu / Debian. TreeSync is Linux only for now.
- **A user that can use `sudo`**, for the system update, which runs every time without asking, and for installing packages (`bsir.sh` shows those commands and asks first).
- **Network access**, to install packages and, if U++ is not on the machine yet, to download it.
- About **2 GB of disk** for U++ and the build output.

1.2 Get the code

```
git clone https://github.com/JayACarlsonAtHome/jac317.git
cd jac317/v001             # on dev-work, the default branch
```

`dev-work` is where the work happens: it is tested on every push, but it is the newest code, not a release. With an SSH key on GitHub, `git@github.com:JayACarlsonAtHome/jac317.git` works too.

Everything below runs from `jac317/v001`.

1.3 See what bsir.sh would do (optional)

```
./bsir.sh --dry-run
```

This checks the machine and prints the plan: which requirements are present, the install command for each missing one, and whether U++ would be fetched. Nothing is updated, installed or built.

1.4 Run bsir.sh

```
./bsir.sh
```

It works in five steps.

Step 1: the Linux family and the system update

It reads `/etc/os-release` and picks the install recipes for `rhel`, `fedora` or `linuxmint` (Ubuntu and Debian count as `linuxmint`). It then runs the system update (`dnf upgrade` or `apt upgrade`) on every run, without asking. If the update fails, it says so and goes on.

Step 2: the requirements

Each requirement in `setup/recipes.conf` is checked and listed as `ok` or `MISSING`:

Name	What it is	Needed for
`gcc`	GCC 15 or newer (RHEL: `gcc-toolset-15`)	building (required)
`clang`	Clang 21 or newer	building (required)
`gui`	GTK 3 and the libraries U++ GUIs build with, OpenSSL for SSH	building (required)
`tools`	make, git, curl, tar, xz	fetching and building U++ (required)
`uppfix`	A one-line fix to U++'s heap (gcc release builds crash without it)	building (required)
`testing`	bc and OpenSSH client and server	the compiler comparison and the SFTP tests (optional)
`checks`	AddressSanitizer + UBSan runtimes and valgrind	the checked test runs (optional)

For the missing ones it shows the install commands, for example on RHEL:

```
To install (these change the system; most need sudo):
  gcc: sudo dnf install -y gcc-toolset-15
  gui: sudo dnf install -y gtk3-devel libnotify-devel ...

Run these now? [y/N]
```

Answer `y` to run them (you will be asked for your `sudo` password), or `n` to stop and run them by hand. If a required one is still missing afterwards, bsir.sh stops here.

Step 3: U++

TreeSync is built with U++ (its build tool `umk` and its sources `uppsrc`). `bsir.sh` looks for one in this order:

1. `$UPP_DIR` (if you set it)
2. `upp_basefix_install` beside this repository: an install made by upp_basefix, U++ pinned to a GitHub commit plus fixes (0 warnings with gcc and clang, sanitizer and valgrind clean, the heap fix below included)

Each is taken as a pair (its `umk` with its own `uppsrc`). If neither is found it offers to install U++ through upp_basefix. Everything goes in the folder this repository was cloned into, so a clone in `~/git` gets `~/git/upp_basefix` (the clone), `~/git/upp_basefix_work` (the download and the trees) and `~/git/upp_basefix_install`. It clones upp_basefix, runs its bootstrap (the pinned U++ from GitHub, `umk` built from it, the fixes; it lists the system packages U++ needs and installs them only when you say yes, with `sudo`), then `upp_basefix.sh install`. To put the install somewhere else:

```
./bsir.sh --upp-dir /opt/upp_basefix
export UPP_DIR=/opt/upp_basefix   # then build.sh finds it too (put this in ~/.bashrc)
```

Step 4: build and test

For each compiler it runs `./build.sh --compiler <gcc|clang> --test`: a **debug** build of the three programs and the core tests. The output of each goes to `out/bsir-gcc.txt` and `out/bsir-clang.txt`.

It puts **TreeSyncBuild** and **TreeSyncInstall** in the **TreeSyncApp** folder of your app list (`setup/app-folder.sh`; see 2.6).

It also points git at the project's pre-push hook (`git config core.hooksPath v001/setup/hooks`), so every `git push` first runs the full check (see 2.5).

Step 5: summary

It ends with one line per step in the start : stop : duration layout:

```
 Start    : Stop     : Duration         : Command / Comments
 14:02:58 : 14:03:01 : 000:000:00:00:03 : Check requirements........ : Result = ....0 : Good
 14:03:01 : 14:05:40 : 000:000:00:02:39 : Build and test with gcc... : Result = ....0 : Good
: passed: 290/290 checks
```

The same lines are added to `Bsir-Results.txt` in TreeSync's logs folder.

1.5 `bsir.sh` options

Option	What it does
`--dry-run`	Only say what it would do
`--yes`	Do not ask before installing (for scripts)
`--compiler gcc` / `clang` / `both`	Build with one compiler (default both)
`--no-build`	Stop after the checks and installs
`--upp-dir DIR`	Where U++ goes if it has to be fetched (default `upp_basefix_install` beside this repository)
`--check`	Only check, one `@req` line per requirement (TreeSyncBuild uses this); exit 1 if a required one is missing
`-h`, `--help`	Show the help

1.6 After bsir.sh

`bsir.sh` leaves **debug** builds in `out/gcc/` and `out/clang/`. For everyday use build the faster **release** programs, and install them:

1. **TreeSyncBuild** (TreeSyncApp folder): Type *Release*, **Build** (or `./verify.sh`: debug and release, all tests, both compilers; ends with release builds).
2. **TreeSyncInstall**: **Install GCC...** and / or **Install Clang...** copies that build to `/usr/local/bin` for root (asks for your password; or `./install.sh`). TreeSyncRoot-GCC, TreeSyncHome-GCC (and

the Clang ones) then join the TreeSyncApp folder.

Most syncs need root, and root runs the installed copy: see 2.6 TreeSyncInstall. Then go on to 3. Run.

2. Build

All building goes through `v001/build.sh`; TreeSyncBuild, `verify.sh` and `bsir.sh` all call it. Run everything from `jac317/v001`.

TreeSync is four programs, all in the **TreeSyncApp** folder of your app list:

Program	Job	Lives in	Runs as
TreeSyncBuild	build and test, with bsir's requirements check (2.3)	the source tree	you
TreeSyncInstall	install the release build for root, or remove it (2.6)	the source tree	you (asks for the password to install)
TreeSyncRoot-GCC, **TreeSyncRoot-Clang**	sync as root, any folder (3.9)	`/usr/local/bin`	you; each sync as root
TreeSyncHome-GCC, **TreeSyncHome-Clang**	sync as you, inside your home (3.1)	`/usr/local/bin`	you

TreeSync is built with both GCC and Clang, and both builds can be installed side by side: each sync program comes once per compiler, and you install one or both.

2.1 What gets built

Program	What it is
`treesync-root`	**TreeSyncRoot**: the sync window, every sync as root
`treesync-home`	**TreeSyncHome**: the same window, every sync as you, inside your home
`treesync-cli`	The command-line program; does every sync (both GUIs start it)
`treesync-build`	**TreeSyncBuild**: builds and tests TreeSync (2.3); never installed itself
`treesync-install`	**TreeSyncInstall**: installs the release build for root (2.6); never installed itself
`treesync-test`	The core tests (built with `--test`)

Each compiler has its own folder: `out/gcc/` and `out/clang/`. The two builds are the same program; TreeSync is also a proving ground for GCC 15+ and Clang 21+ with C++26 (RHEL: gcc-toolset-15; Fedora 44: GCC 16 and Clang 22).

2.2 build.sh

```
./build.sh --compiler gcc                # debug build with gcc (15 or newer)
./build.sh --compiler clang              # debug build with clang (21 or newer; the default
compiler)
./build.sh --compiler gcc --release      # optimized build: use this for real syncs
./build.sh --compiler gcc --test         # build, then run the core tests
./build.sh --compiler gcc --release --test
./build.sh --compiler gcc --clean        # compile everything, not only what changed
```

Option	What it does
`--compiler gcc` / `clang`	Which compiler (default clang)
`--release`	Optimized build (default: debug)
`--test`	Also build `treesync-test` and run it; the exit code is non-zero if a test fails
`--clean`	Rebuild everything (fair build times when comparing compilers)
`--tsan`	Instead of the build: the tests under ThreadSanitizer (clang), into `out/clang-tsan/`
`--asan`	The tests under AddressSanitizer + UBSan, then `treesync-cli --bench` under them
`--valgrind`	The tests under valgrind memcheck (slow)
`--helgrind`	The tests under valgrind helgrind (very slow)

Format checks. Every package's `.upp` has `options "-Wformat=2 -Werror=format"`, so a printf-style call whose arguments do not match its format (for example a 64-bit value given to `%d`) stops the build instead of misreading memory at run time. The flags are package options, so they cover TreeSync's code and not U++'s own. The toolchain report (**This program was built with** in TreeSyncBuild, and `treesync-cli --toolchain`) says `Formats: checked` when they were applied.

Debug or release? Debug builds check more and are easier to debug, but are much slower: on many small files about 4 times the CPU. Always run real syncs with a **release** build. `build.sh` remembers which type is in each `out/` folder and links the programs again when you switch, so a debug build never leaves a release program behind or the other way round.

A test run ends with a line such as `passed: 348/348 checks`. A debug build skips one test (a 4.5 GiB sparse copy, too slow without optimization) and says so; a release test run has all of them (`passed: 355/355 checks`, 2026-10-19).

2.3 TreeSyncBuild

Building and testing have a program of their own, separate from the ones that sync. Start it as yourself: **TreeSyncBuild** in the TreeSyncApp folder of your app list (`bsir.sh` puts it there), or `v001/out/gcc/treesync-build`. It lives in the source tree and is never installed; as root it refuses to start

(root runs TreeSync, it does not build it). It also does bsir's part on a machine that has been set up: **This machine** runs `bsir.sh --check`, and **Open terminal to fix...** opens `bsir.sh` in a terminal.

- **Compiler**: gcc, clang, or both one after the other.
- **Type**: Debug or Release.

- **Run the tests (treesync-test) after building**: the one switch for testing. Unticked, **Build** only builds. Ticked, the tests run, plus the checked runs below it:

    - Also under ThreadSanitizer (clang; data races)
    - Also under AddressSanitizer + UBSan (memory errors, undefined behaviour)
    - Also under valgrind memcheck (slow)
    - Also under valgrind helgrind (very slow)

    These keep their ticks while "Run the tests" is off; they are greyed out and skipped. - **Clean build**: compile everything. - **Compare the compilers**: versions, C++26 features, `import std`, speed and size, side by side. - **Open TreeSyncInstall after a good release build**: when every step went well, TreeSyncInstall (2.6) opens, ready to install what was just built. Only for a release build (a debug build is not installed). - **Display**: Wayland (recommended) or X11, for all four programs. - **This machine**: whether the requirements are present (**Check again**, **Open terminal to fix...**).

The command lines it will run are shown at the bottom. The buttons:

- **Build**: runs them one after another and shows each one's output; the results screen sums them up (kept in `Build-Results.txt` in the logs folder).
- **Install...**: opens TreeSyncInstall (2.6), for the builds already in `out/`.
- **Run TreeSyncHome-GCC (just built) / Run TreeSyncHome-Clang (just built)**: starts the TreeSyncHome this window just built (from `v001/out/gcc` or `out/clang`, not the installed one), as you, in a window of its own, to try it before installing; TreeSyncBuild stays open.
- **Help** (F1): this chapter.

Your choices are saved in `~/.config/treesync/build`.

2.4 The sanitizer and valgrind runs

These find bugs the normal tests cannot see: data races between the copy threads, memory errors, undefined behaviour. They take much longer than the normal tests, so run them when threading or memory handling changed:

```
./build.sh --compiler clang --tsan
./build.sh --compiler gcc --asan
./build.sh --compiler gcc --valgrind
./build.sh --compiler gcc --helgrind
```

They need the optional `checks` requirement (see 1.4). Each builds into its own folder (`out/gcc-asan/` and so on) and never touches the programs in `out/gcc/`.

2.5 verify.sh: the full check, and the pre-push hook

```
./verify.sh                 # gcc and clang
./verify.sh --compiler gcc  # one compiler
```

For each compiler it does a **debug build with every test**, then a **release build with every test**. It stops at the first failure and ends with **release builds in `out/`**, the programs the GUI runs. Each step's full output is in `out/verify/<compiler>-<type>.log`. It also checks that each step really tested the type it claims. A run takes well under a minute when little has changed:

```
verify: debug, then release; every test; compilers: gcc clang
 gcc    debug    building and testing ... ok    10 s  passed: 296/296 checks
 clang  debug    building and testing ... ok     7 s  passed: 296/296 checks
 gcc    release  building and testing ... ok    13 s  passed: 303/303 checks
 clang  release  building and testing ... ok    13 s  passed: 303/303 checks
 took 43 s
 verify: all good; out/ holds release builds
```

The pre-push hook. `v001/setup/hooks/pre-push` runs `verify.sh` before every `git push`; a failure stops the push. `bsir.sh` sets it up; by hand:

```
git config core.hooksPath v001/setup/hooks
```

To push once without the check: `git push --no-verify`.

2.6 Install for root: TreeSyncInstall

You build as yourself; most syncs need root. Root must not run programs your user can change (anything running as you could swap them first), so the release programs are **installed** into the usual system folders, owned by root.

TreeSyncInstall (in the TreeSyncApp folder, or `v001/out/gcc/treesync-install`) does it. It runs as you, from the source tree. It has **one row per compiler**, GCC and Clang, each installed on its own, so you can install one or both:

- What the build in `out/` is: debug or release, and when it was built. Only a release build can be installed.
- Whether it is installed: *not installed*, *installed: this build*, or *installed: an older build* (then Install gives root the new one).
- **Install GCC...** / **Install Clang...**: runs `install.sh --compiler gcc` (or `clang`) as root (it asks for your password once per window: later Installs and Removes in the same window do not ask again), then puts the TreeSyncApp folder in your app list (`setup/app-folder.sh`). The other compiler's install stays as it is.
- **Remove GCC...** / **Remove Clang...**: removes that one; the other stays.

Below: **Installed now** (what `INSTALLED-gcc` and `INSTALLED-clang` say: build, commit, who, when) and the output of `install.sh`. **Show what it does** shows a dry run (no password). **Remove all...** removes everything it installed; your settings, databases and logs stay. **Help** (F1): this chapter. As root it refuses to start.

The same from a terminal:

```
./install.sh                              # both release builds; asks for your sudo
password
./install.sh --compiler gcc               # only the GCC one (or clang); the other stays
./install.sh --dry-run                    # only say what it would do
./install.sh --uninstall --compiler clang # remove only the Clang one
./install.sh --uninstall                  # remove it all again
```

Installed	Where
The programs	`/usr/local/bin/treesync-root-gcc`, `treesync-home-gcc`, `treesync-cli-gcc`, and the same with `-clang` (`root:root`, mode 755, like every program in `/usr/bin`)
`treesync-cli` for the terminal	`/usr/local/bin/treesync-cli`: a link to `treesync-cli-gcc`, or to `-clang` when only that one is installed
The manual	`/usr/local/share/doc/treesync/pdf/` (the Help menu finds it there)
What was installed	`/usr/local/share/doc/treesync/INSTALLED-gcc`, `INSTALLED-clang`: build, commit, who, when, the SHA-256 of its `treesync-cli`
App list entries	`/usr/local/share/applications/treesync-root-gcc.desktop`, `treesync-home-gcc.desktop` (and `-clang`): **TreeSyncRoot-GCC**, **TreeSyncHome-GCC**, **TreeSyncRoot-Clang**, **TreeSyncHome-Clang**, in the TreeSyncApp folder
polkit actions	`/usr/share/polkit-1/actions/org.treesync.run-as-root.policy`, one per compiler: TreeSyncRoot asks for the password once per window, and a new window within a few minutes does not ask again

Each sync program runs the `treesync-cli` of its own compiler: TreeSyncRoot-Clang never runs the GCC engine. Both builds share your settings, database and logs.

It refuses a debug build: build a release in TreeSyncBuild (or `./verify.sh`, or `./build.sh --compiler gcc --release`) first. Programs go in under a temporary name and are renamed into place, so a TreeSync that is running keeps its copy. Older installs' programs and app entries without the compiler in their names (`treesync`, `treesync-root`, `treesync-home`) are removed.

Install again after every build you want to use as root. The installed copy does not follow `out/` by itself.

The installed copy only runs. TreeSyncBuild and TreeSyncInstall stay in the source tree. TreeSyncRoot syncs as root, for administrators; TreeSyncHome syncs as you, for anyone (see 3.9).

The TreeSyncApp folder. Every entry carries the category `X-TreeSyncApp`. `setup/app-folder.sh` (run by bsir.sh and TreeSyncInstall, as you) writes the TreeSyncBuild and TreeSyncInstall entries in `~/.local/share/applications` and makes the folder: on GNOME an app folder in the app grid (`gsettings org.gnome.desktop.app-folders`), on other desktops a menu folder (`~/.config/menus/applications-merged/treesyncapp.menu`). `setup/app-folder.sh --remove` takes it away again. Each user who wants the folder runs it once; without it the entries are still in the app list, just not grouped.

2.7 Building as root

You do not need to, and TreeSyncBuild and TreeSyncInstall refuse to start as root. The scripts still work as root, through `sudo`, `pkexec` or a root login: they use the U++ of the user the TreeSync folder belongs to (root's home has none), and give everything they made in `out/` back to that user, so your own next build can overwrite it.

2.8 After changing code

1. Build and test while working: `./build.sh --compiler gcc --test`.
2. Before pushing (or let the hook do it): `./verify.sh`.
3. When threads, locks or memory handling changed, also the sanitizer runs (2.4).
4. To use the new build as root: TreeSyncInstall (or `./install.sh`).

`verify.sh` leaves release builds in `out/`, so the GUI is fast again afterwards.

3. Run

3.1 Start the GUI: TreeSyncRoot or TreeSyncHome

Two programs sync, with the same window; they differ in who the syncs run as:

	TreeSyncRoot	**TreeSyncHome**
Start it	**TreeSyncRoot-GCC** or **-Clang** in the TreeSyncApp folder, or `treesync-root-gcc`	**TreeSyncHome-GCC** or **-Clang** in the TreeSyncApp folder, or `treesync-home-gcc`
Syncs as	**root**, each run through `pkexec`	you
Paths	any	only inside your home
Who	administrators (root, `wheel`, `sudo`, `admin`); anyone else is told to use TreeSyncHome	anyone; it refuses to start as root
Settings	`~/.config/treesync/settings`	`~/.config/treesync/settings-home`

Each comes once per compiler, GCC and Clang; TreeSyncInstall installs one or both (see 2.6). They do the same; both builds share the settings, database and logs. For everyday syncs use the **installed** ones. The builds in the source tree (`v001/out/gcc/treesync-home`, `treesync-root`) are for working on TreeSync itself; **Run TreeSyncHome-GCC (just built)** in TreeSyncBuild starts one. Neither has anything to do with building: that is **TreeSyncBuild** (see 2.3).

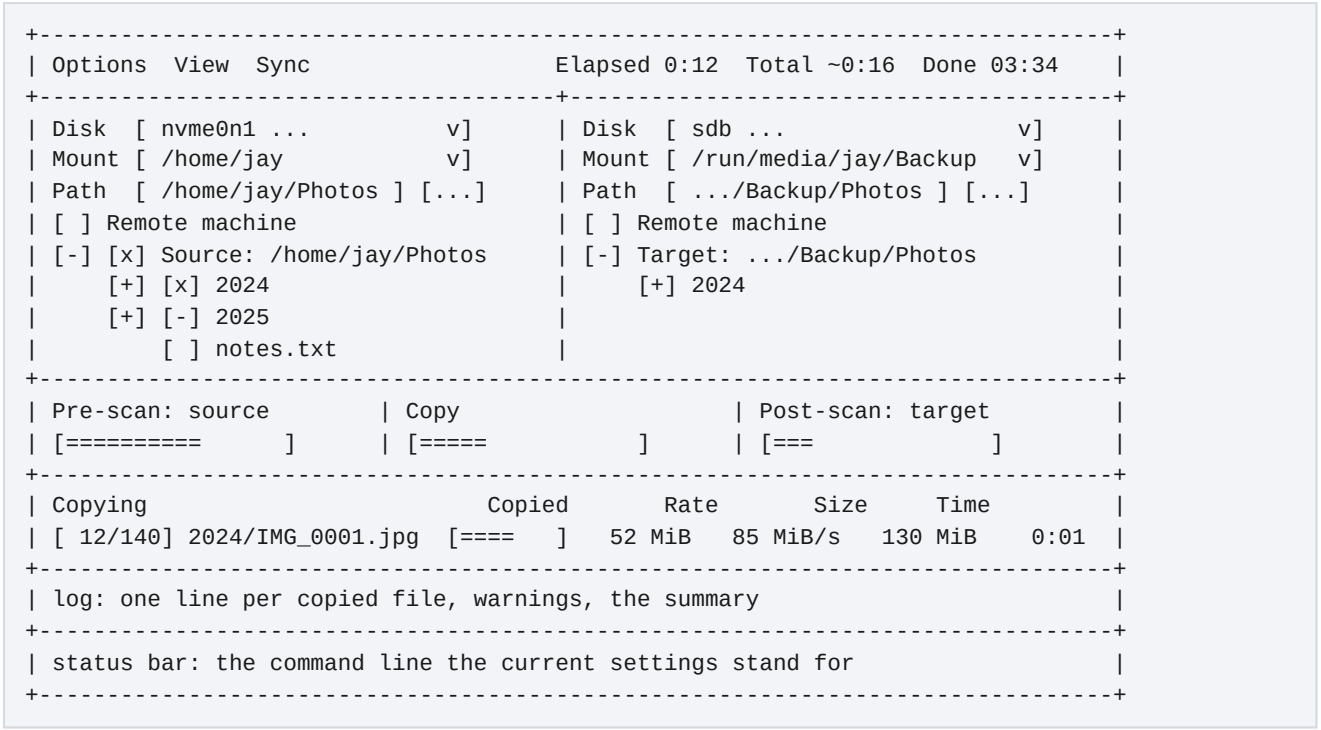
Always start the GUI as yourself, not with `sudo`: it is an ordinary desktop program, and only TreeSyncRoot's syncs run as root.

Options on the command line set up the window and become the saved settings:

```
treesync-root-gcc --source=/home/jay/Photos --target=/run/media/jay/Backup/Photos --mirror
```

Without options, the settings saved last time are used (each program its own file, see the table; saved on every change).

3.2 The main window



- **Left: the source**, **right: the target**. Each side has a **Disk** and **Mount** list to jump to a drive, a **Path** box (type a path and press Enter, or **...** to browse), **Remote machine** for a tree on another machine (3.10), and the tree below it. TreeSyncRoot lists every disk; TreeSyncHome lists only your home (the title bar says which window this is).
- **Trees**: click [+] to open a folder, or use the + and - keys. **View > Reload trees** reads them again, **View > Collapse all** closes them.

- **Right-click** a file or folder in either tree to change it by hand. The window then reads the trees again:

    - **New folder...**: a folder inside the one you clicked (or next to the file you clicked).
    - **Rename...**: a new name in the same folder. It never replaces something that is already there.
    - **Delete...**: moves it to the `.old` folder, like the files a sync deletes. It can be brought back from there until **Clear .old**.
    - **Delete for good...**: deletes it, and for a folder everything in it. This cannot be undone.

    The changes are made the way a sync is: as root in TreeSyncRoot (with the same one password per window), as you inside your home in TreeSyncHome, over SFTP for a remote tree. They are logged like a run (`Edit (delete)` in Results.txt). The menu is greyed out while a sync runs, and TreeSync's own files (`.treesync-old`, `*.treesync-part`) and the top folder itself cannot be changed. Changing the **source** also changes what the next run does: a file deleted from the source is removed from the target by the next Mirror. - **Free space**: under each tree, a bar and a line show the disk's free space, e.g. `384.0 GiB free of 930.0 GiB (58% used)`. It turns red under 5% free. Local trees show it all the time. A remote tree shows it once a run has asked its server. During a run it updates after every file, at most once a second. Between the disk's own figures, at the start and the end of the run, TreeSync works it out from what the copies write (holes count as nothing), shown with a `~`. So another program filling the disk at the same time only shows at the end. - **The progress boxes** and the list of files in progress appear while a run is going. Each file in progress

shows a bar, **Copied** (the bytes copied so far; **Read** in a Scan only run), its **Rate**, its **Size** and its **Time** so far. When idle threads help with a big file (*Split big files across idle threads*), its row counts what all of them copied, and the log says *splitting NAME*. The upper right of the menu bar shows how long it has run, how long the whole run will probably take, and when it should be done. - **The log** shows what happened; **View > Show log file** (Ctrl+L) opens the full log of the newest run.

Without root, local paths are limited to your home folder; the status bar says so.

Help opens this manual: the whole of it, or one chapter, as a PDF. **F1** opens the chapter for the window you are in.

3.3 Choose what to sync: the checkboxes

Every folder and file in the **source** tree has a checkbox.

- **Check a folder**: everything below it is synced, including files added there later.
- **Uncheck a folder**: nothing below it is synced.
- **Uncheck one file** inside a checked folder: the rest of that folder stays checked.
- A folder with only some of its contents checked shows a **dash** (partly checked).
- The **top box** checks or unchecks the whole tree.

What is checked is saved as `--include-path` settings. Folders above a checked item are created on the target only where something checked goes, and **Mirror never deletes anything outside the checked paths**. Changing the source folder starts again with everything checked. If nothing is checked, Run refuses to start.

3.4 Options: mode and switches

Options (in the menu bar) opens the sync's mode and switches. **OK** keeps the changes, **Cancel** throws them away. The trees and their remote logins are not here: they are on the main window (3.2, 3.10).

- **Mode**:
  - **Copy**: new and changed files go from source to target; nothing on the target is removed. Turns on *Skip older*: a newer file on the target is not overwritten.
  - **Mirror**: the target is made to match the source; files only the target has are removed (with safe delete: moved to `<target>.old`, or to `<target>/.treesync-old` when the target is the top folder of a disk, such as a USB drive).
  - **Merge**: changes go both ways; a file changed on both sides since the last sync is a conflict and is left alone. Nothing is removed.
- **Reset to the defaults** of Copy, Mirror or Merge.
- **Owners**: don't copy / by name (normal) / by number. Setting an owner needs root.
- **Switches**: click any option to see what it does in the help box. The important ones: *Safe delete* (on; keep it on), *Hard links*, *Follow links*, *Skip older / newer / changed*, *Keep extra*, *Wipe target first*, *Check there is room before copying* (on; see 3.5), *Low priority*, *Split big files across idle threads* (on: a thread with no file left to start copies chunks of a big file another thread is on, so the last big file of a run is not left to one slow stream).
- **Numbers**: threads (default 4), retries, chunk size, bandwidth limit, how long to wait for a lost network, time tolerance.

- **Include / exclude**: file and folder names with `*` and `?`, separated by `;`. A blank box filters nothing; exclude wins over include.

3.5 Dry run, then run

Room check first. Before a run copies anything, it lists both trees (quickly, without reading files) and adds up the most the copies can need: new and changed files at their full size. If that, plus a reserve, is more than the target disk has free, the run does not start (exit code 4) and the GUI shows the numbers and asks whether to start anyway. A dry run shows the same check, in the log and in its popup at the end (green, or red when it may not fit), e.g. *room check: the copies need at most 2.1 GiB (14 files); the target disk has 457.1 GiB free: room to copy*.

The check cannot be exact. A file with long runs of zeros (a VM image) takes much less disk than its size: the 400 GiB image above takes 53 GiB. That is only known when it is read, so **start anyway** is often right for VM images. Either way, a run that gets close to filling the target **stops by itself**, before the disk is full in the middle of a file. It keeps a reserve free: 1% of the disk, at least 256 MiB (or 5% of a disk too small for that), at most 4 GiB. The log says *stopping: the target disk is nearly full*, and once you have made room, **Run** continues where it stopped. The switch **Check there is room before copying** (`--no-precheck`) turns the check off; the stop near full stays on.

1. **Sync > Dry run** (Shift+F9): lists what a run would do (`copy`, `link`, `delete`, `attrs`, `mkdir`) and changes nothing. Read it.
2. **Sync > Run** (F9): asks for confirmation, saying what the mode will do and where removed and replaced files go, then runs.

While it runs:

- **Pre-scan: source** walks the source. **Copy** shows the bytes copied of the total. **Post-scan: target** checks each target folder for files to remove (Mirror).
- Each file is written to `NAME.treesync-part`, read back, checked against the source's SHA-256, given the source's permissions, times, owner and extended attributes, and only then renamed into place.
- A file whose content is the same but whose permissions or time changed gets only its attributes set (an `attrs` step), without copying.

At the end a popup says how it went and how long it took. **Sync > Stop** (Esc) stops after the current file or chunk.

When files were copied, the popup and the end of the log also show the **check**: the numbers that prove the copies are the source.

```
check: 1 file copied and read back; its SHA-256 matches the source's
  bytes: 429,562,527,744 on the source, 429,562,527,744 in the copies: the same
  this run copied 381.3 GiB; the other 18.8 GiB (...) were copied by the stopped run
    before it, read back again and kept
  on disk the copies take 53.0 GiB for 400.1 GiB of data: runs of zeros are kept as
    holes (sparse), which read back as zeros, so the content and the SHA-256 are the same
  mode, time and owner: kept
  the file:
    Fedora_Base.qcow2
      source  429,562,527,744 bytes  SHA-256 0256ebac...
      copy    429,562,527,744 bytes  SHA-256 0256ebac...  MATCH
      time    2026-09-19 23:38:18.273011253 on both
      mode    600 on both;  owner root:libvirt on both
```

Every byte is counted on both sides, and the SHA-256 of each copy (read back from the target) is shown next to the source's. With many files, the totals come first, then the 10 biggest in full. A number that differs on purpose says why where it is shown: - **Resumed run:** it copies fewer bytes than the file has, because a stopped run had already copied the rest, and that part was checked again. - **Holes:** a copy can take less disk than its size (`du` shows less than `ls -l`). Runs of zeros are kept as holes. Results.txt gets the totals on the run's line, e.g. *1 file checked: 429,562,527,744 bytes on both sides, SHA-256 match*.

3.6 Stop and resume

A stopped run (Stop, Ctrl+C in a terminal, a crash, a reboot, an unplugged disk) is **resumed by running it again with the same settings**. Every step was recorded in the database, so it continues where it was. A big file continues at its last good chunk: on resume, every chunk already copied is read back and checked first, and from the first damaged one the file is copied again.

To start over instead, use `--fresh` on the command line.

3.7 Checking and attributes

- **Sync > Scan only (SHA-256 check)** (F7): changes nothing. Lists both trees, reads and hashes **every file on both sides** afresh, and compares permissions, times, owners and extended attributes. Each file's log line ends in `same`, `DIFFERS`, `missing` (not on the target) or `failed`. Use it after a big copy to prove the target is right.
- **Sync > Sync attributes** (F8): sets only the attributes that differ (mode, time, owner, extended attributes) on the target, for files of the same size and for folders. No content is copied.
- **Sync > Clear .old folders...**: deletes the safe-delete folders (`<target>.old`) for good, after a confirmation that says how many files and bytes. Folders made by a root run need **TreeSyncRoot**. On the command line, without `--no-dry-run` it only counts; its result says *Counted, not deleted*, *Good* (deleted) or *Nothing to clear*.

3.8 History, logs and AI export

- **View > History...** (Ctrl+H): every run, newest first, with its counts (folders, copied, deleted, attributes set, failed, conflicts, bytes). Failed runs are red. Below:
  - **Actions**: what the selected run did, failures first.

    - **SHA-256 checks**: every checked file under a line naming its folder, with the source and target SHA-256 side by side, whether they match, and how the attributes compare. Mismatches first, in red; attribute differences in orange.
    - **Show log**, **Export logs to AI.md**, **Export SQL to AI.md**.
- **Logs**: each run writes `logs/YYYY-MM-DD_HHMMSS.log` next to the database. It starts with a header; every line has its start, stop and duration to the microsecond.
- **View > Export newest run for AI**: writes the run as Markdown files into `logs/ai/` (a guide, problems, slowest copies, throughput, the logs in full; or the database contents), ready to give to an AI to analyse.

3.9 Running as root

A sync often meets files your user cannot read or replace (other users' files, system files), so **TreeSyncRoot syncs as root, for an administrator**: root, or a member of `wheel` (`sudo` or `admin` on Debian, Ubuntu and Mint), the users who may use sudo. A tool that can overwrite any file as root is as powerful as root, so nobody else gets that: for anyone else TreeSyncRoot says so and does not start, and they use **TreeSyncHome**, which syncs as themselves, inside their own home, with no password asked.

The window's title says which one it is and how its syncs run: *TreeSyncRoot-GCC - syncs as root*, or *TreeSyncHome-GCC - syncs as jay, inside /home/jay*. TreeSyncRoot offers every disk; TreeSyncHome only your home.

For an administrator:

- **TreeSyncRoot** runs every sync through `pkexec`, with the installed `treesync-cli-gcc` (or `-clang`: the one of its own compiler), which only root can change. The first run of a window asks for your password, once: that window's later runs go through the same root process (`treesync-cli --root-session`), which ends when the window closes. A window opened within a few minutes does not ask either (polkit remembers it). Any path may be chosen.
- **TreeSyncHome** runs every sync as you (it passes `--home` to `treesync-cli`), so it never asks for a password and stays inside your home.
- **The installed `treesync-cli`**, started from a terminal by an administrator, starts itself again through `sudo`: `treesync-cli --source=... --target=...` runs as root (sudo asks for your password). `--help` and `--home` do not.
- **The source-tree TreeSyncRoot** (`out/gcc/treesync-root`) also uses the **installed** `treesync-cli`. If that is not this build, the log says so (TreeSyncInstall updates it); if nothing is installed, it warns that it runs a copy your user could change.

A root run keeps its own database and logs under `/root/.local/share/treesync/`, separate from yours. To resume a root run, run it as root again.

History and logs of root runs. The GUI's **History** and **Show log file** read *your* database and logs, and root's are not readable by you. To look at a root run: `sudo ls /root/.local/share/treesync/logs/`, or have root export it: `treesync-cli --export-ai-logs` (the installed one runs as root by itself; it writes into root's `logs/ai/`). Note that `sudo treesync-cli` does not work on RHEL: sudo does not search `/usr/local/bin`; the installed `treesync-cli` needs no sudo in front.

3.10 Remote trees (SFTP)

Either side can be on another machine over SSH. One-time setup:

```
ssh-keygen -t ed25519            # if you have no key yet
ssh-copy-id jay@server           # passwordless login
ssh jay@server                   # once, to check and accept the host key
```

TreeSync logs in with keys only, never passwords. In the GUI, on the side that is remote (usually the target), tick **Remote machine** under Path. A login row opens:

```
[x] Remote machine
Host [ server              ]  Port [22]  User [ jay  ]
Key  [                     ]          [Test]  [Done]
```

Fill in Host, Port and User (and Key, if not the usual one), and the Path above it (**...** picks it from the server's folders, starting at `/`). **Test** logs in, lists the path and, when that works, uses the login and folds the row into one line:

```
[x] Remote machine   jay@server:22   (test passed: 12 entries)   [Show / Test]
```

If the test fails it says why and leaves the row open. **Done** uses the login without a test. **Show / Test** opens the row again, to see it, change it or test it again. Unticking **Remote machine** makes the tree local again at once. Until a login is used (Test or Done), the tree and the runs keep the one they had. On the command line: `--target-host=server --target-user=jay`.

If the network drops, the run waits (default up to 30 minutes, *Wait for the network*), shows **NETWORK UNAVAILABLE**, and carries on when the machine is back.

Checking copies on the server. When the login may run commands and the server has `sha256sum`, the server hashes its own files: copies are checked there and remote scans hash there, so nothing is read back over the network. The log says which way it works (`server hashes its own files` or `files are read over SFTP`). An SFTP-only account still works, just with twice the traffic for a verified copy.

3.11 The command line

`treesync-cli` takes the same options the GUI saves. **It does a dry run unless you give `--no-dry-run`.** The examples use the installed `treesync-cli` (it runs as root); in the source tree it is `v001/out/gcc/treesync-cli` (it runs as you).

```
# What would a mirror do?
treesync-cli --source=/home/jay/Photos --target=/run/media/jay/Backup/Photos --mirror

# Do it
treesync-cli --source=/home/jay/Photos --target=/run/media/jay/Backup/Photos --mirror --no-
dry-run

# Copy only new and changed files, 8 threads, at low priority
treesync-cli --source=/data --target=/backup/data --copy --threads=8 --low-priority --no-dry-
run

# Only one folder and one file of the source
treesync-cli --source=/data --target=/backup/data --include-path=projects/2026 --include-
path=notes.txt --no-dry-run

# To a remote machine
treesync-cli --source=/data --target-host=server --target-user=jay --target=/backup/data --
no-dry-run

# Prove a copy: hash both sides, change nothing
treesync-cli --source=/data --target=/backup/data --scan-only

# Set only differing permissions and times
treesync-cli --source=/data --target=/backup/data --sync-attrs --no-dry-run

# Delete <target>.old for good (without --no-dry-run it only counts)
treesync-cli --source=/data --target=/backup/data --clear-old --no-dry-run

# One change by hand (what the right-click menu does): mkdir, rename, delete
# (to the .old folder) or delete-for-good
treesync-cli --source=/data --target=/backup/data --edit=rename --edit-side=target \
             --edit-path=old/name.txt --edit-name=new-name.txt
```

`treesync-cli --help` lists every option. In a terminal it shows one line per copied file, `[ 6/90] name 100% | 85.3 MiB/s | 130.9 MiB | 00m01s`, and Ctrl+C stops it (press twice to quit at once).

Exit codes: 0 in sync (or, for Scan only, the trees are the same); 1 failures, conflicts or differences; 2 could not run (bad options, a path refused); 3 stopped (also by itself, when the target disk was nearly full); 4 may not fit (the room check; nothing was copied, `--no-precheck` starts anyway).

Database views for looking at results directly:

```
sqlite3 ~/.local/share/treesync/treesync.db
```

```
SELECT run, started, state, copied, failed FROM run_summary ORDER BY run DESC LIMIT 10;
SELECT * FROM failures WHERE run = 42;
SELECT path, file_name, problem FROM hash_mismatches WHERE run = 42;
SELECT path, file_name, attr_diff, result FROM attr_mismatches WHERE run = 42;
```

4. Files and troubleshooting

4.1 Where everything is kept

What	Where
Installed programs	`/usr/local/bin/treesync-root-gcc`, `treesync-home-gcc`, `treesync-cli-gcc`, the same with `-clang`, and `treesync-cli` (a link) (`root:root`, 755; by TreeSyncInstall / `install.sh`)
Installed manual	`/usr/local/share/doc/treesync/pdf/`, and `INSTALLED-gcc`, `INSTALLED-clang`: which build, commit, when
App entries, installed	`/usr/local/share/applications/treesync-root-gcc.desktop`, `treesync-home-gcc.desktop`, and the same with `-clang`
polkit action	`/usr/share/polkit-1/actions/org.treesync.run-as-root.policy`
Built programs	`v001/out/gcc/` and `v001/out/clang/`: `treesync-root`, `treesync-home`, `treesync-cli`, `treesync-build`, `treesync-install`, `treesync-test`
Build and test output	`v001/out/` (`verify/` logs, `bsir-*.txt`); ignored by git
GUI settings	`~/.config/treesync/settings` (TreeSyncRoot), `settings-home` (TreeSyncHome): one option per line, as on the command line
TreeSyncBuild's choices (and the display)	`~/.config/treesync/build`
TreeSyncBuild and TreeSyncInstall in your app list	`~/.local/share/applications/treesync-build.desktop`, `treesync-install.desktop` (by `setup/app-folder.sh`)
The TreeSyncApp folder	GNOME: `gsettings get org.gnome.desktop.app-folders folder-children`; other desktops: `~/.config/menus/applications-merged/treesyncapp.menu`
Database	`~/.local/share/treesync/treesync.db` (SQLite): runs, actions, hash cache, checks
Run logs	`~/.local/share/treesync/logs/YYYY-MM-DD_HHMMSS.log`
Results of every run	`~/.local/share/treesync/logs/Results.txt`, one line per run
AI exports	`~/.local/share/treesync/logs/ai/run-N-logs.AI.md`, `run-N-sql.AI.md`
Root runs	the same under `/root/.local/share/treesync/`
Safe-delete folder	`<target>.old` next to the target; `<target>/.treesync-old` inside it when the target is the top folder of a disk (or `--safe-delete-dir`)
Unfinished copies	`NAME.treesync-part` next to where the file goes; removed when done

What	Where
This manual	`Manual/*.md` (the sources) and `Manual/pdf/` (built by `Manual/build-pdf.sh`)

`--db=PATH` puts the database (and its `logs/` folder) somewhere else.

4.2 Troubleshooting

"... is outside your home directory. Without root, TreeSync only works inside ..."
That is TreeSyncHome (or `treesync-cli` running as you). To sync other folders use **TreeSyncRoot**, which syncs as root for administrators. Inside your home nothing is needed.

"TreeSyncRoot is for administrators"
You are not an administrator, so you sync with TreeSyncHome, as yourself, inside your home. To sync other folders, an administrator does it, or adds you to wheel (`sudo usermod -aG wheel NAME`; log in again), which also lets you use sudo.

TreeSyncRoot and TreeSyncHome are not in the app list, or not in the TreeSyncApp folder
They are installed by TreeSyncInstall; until then only TreeSyncBuild and TreeSyncInstall are there. The folder is per user: run `v001/setup/app-folder.sh` as yourself (TreeSyncInstall and bsir.sh do). GNOME may need you to log out and in once to show a new folder.

Root runs use an older TreeSync, or the log says the installed one "is not this build"
The installed copy does not follow `out/`: install again with TreeSyncInstall (or `./install.sh`) after building. TreeSyncInstall shows which build is installed, and so does `cat /usr/local/share/doc/treesync/INSTALLED-gcc` (or `-clang`).

The password is asked at every run
TreeSyncRoot asks once per window; if each new window asks at once, the polkit action is missing: it is installed only when `install.sh` runs as root (the default, through sudo), not for a `--prefix` of your own.

`sudo treesync-cli`: "command not found"
sudo does not search `/usr/local/bin` on RHEL. Leave the sudo out: the installed `treesync-cli` starts itself through sudo.

TreeSync is not in the app list
Run `./install.sh`; log out and in again if the desktop does not pick it up at once.

A path is refused
Without root, a path that leaves your home (also through `..` or a symlink) is refused. Use root, or a path inside your home.

bsir.sh stops: "Still missing: ..."
A required package did not install. Run the install command it printed by hand and read its error (often a repository that is not enabled, for example CodeReady Builder on RHEL for `gtk3-devel`), then run `./bsir.sh` again.

build.sh: "U++ (umk + uppsrc) not found"
Run `./bsir.sh`, or point to an existing U++ with `export UPP_DIR=/path/to/u++`.

gcc build fails to link: "undefined reference to `handle_contract_violation`"

GCC 16 (Fedora 44) has native C++26 contracts and keeps their default violation handler in `libstdc++exp`, which is linked only with `-fcontracts`. `build.sh` adds it for GCC 16 and newer (not for GCC 15, where `-fcontracts` means something else). An older checkout, or a build method of your own, does not: update TreeSync, or add `-fcontracts` to the build method's link options.

Building as root fails: "U++ not found", or umk "error while loading shared libraries"

Older scripts looked for U++ in root's home and then picked a U++ built for another distribution. Current ones use the U++ of the folder's owner (`~owner/u++`) and skip one that cannot run here. Root-owned leftovers in `out/` from such a build: `sudo chown -R $USER v001/out`.

A gcc release build crashes at start

The U++ heap fix is missing (`uppfix`); `./bsir.sh` applies it.

A test fails

`./verify.sh` stops and prints the failing checks; the full output is in `v001/out/verify/<compiler>-<type>.log`. `git push --no-verify` pushes anyway, but fix it first if you can.

A run is slow

Make sure the GUI runs a **release** build (`./verify.sh` leaves them in `out/`). Many small files: more threads (`--threads=8`) can help. Over SFTP one stream is often slow while several together are not (each waits for its writes and its checks); with *Split big files across idle threads* on (normal) every thread works on the last big files, for a local source. `--compress` helps text, not media.

"no Wayland session found" in TreeSyncBuild

The GUI was started without `WAYLAND_DISPLAY`, usually through `sudo`. Start it as yourself, and use TreeSyncRoot for root runs.

The host key is refused (remote trees)

Connect once with `ssh user@host` to check and accept the key; TreeSync never accepts an unknown or changed key by itself.

"the copies may not fit" (exit code 4)

The room check found that the new and changed files, at full size, are more than the target disk has free. Make room (**Clear .old** frees what earlier runs moved aside), or start anyway: files with runs of zeros (VM images) need much less than their size, and the run stops by itself before the disk is full.

"stopping: the target disk is nearly full"

The run stopped itself to keep a reserve free (see 3.5). Nothing is lost: make room, then **Run** again and it continues where it stopped.

A right-click change says "Not changed"

The message says why: a name that already exists (Rename never replaces), a name TreeSync keeps for itself, or no permission (TreeSyncHome changes only what you may change; TreeSyncRoot changes anything).

The copy is smaller than the source in `du`, or in the disk's free space

Runs of zeros (common in VM images) are kept as holes, so the copy takes less disk than its size. `ls -l` shows the same size on both sides, and so does the check at the end of the run, with the same SHA-256 on both sides. The check says so in its "on disk" line.

A run says it copied fewer bytes than the file has

It resumed a stopped run. The bytes the stopped run had copied were read back, checked and kept, so only the rest was copied. The check at the end says how much came from the stopped run, and shows both sides' full size and SHA-256.

A run was stopped and the target has `.treesync-part` files

Run the same sync again: it resumes, re-checks what was already copied, and removes the part files when done. `--fresh` starts over instead.

"cannot move ... it is on a different disk"

A move cannot cross disks. A target at the top of a disk gets its safe-delete folder inside it (`.treesync-old`) by itself; this message means `--safe-delete-dir` points to another disk. Use a folder on the target's disk, or no `--safe-delete-dir`.

The `.old` folder keeps growing

Safe delete never deletes. **Sync > Clear .old folders...** (or `treesync-cli --clear-old --no-dry-run`) empties it when you are sure.

Something else

View > Export newest run for AI and give the files in `logs/ai/` to an AI, or read them yourself: they list every problem of the run first.